

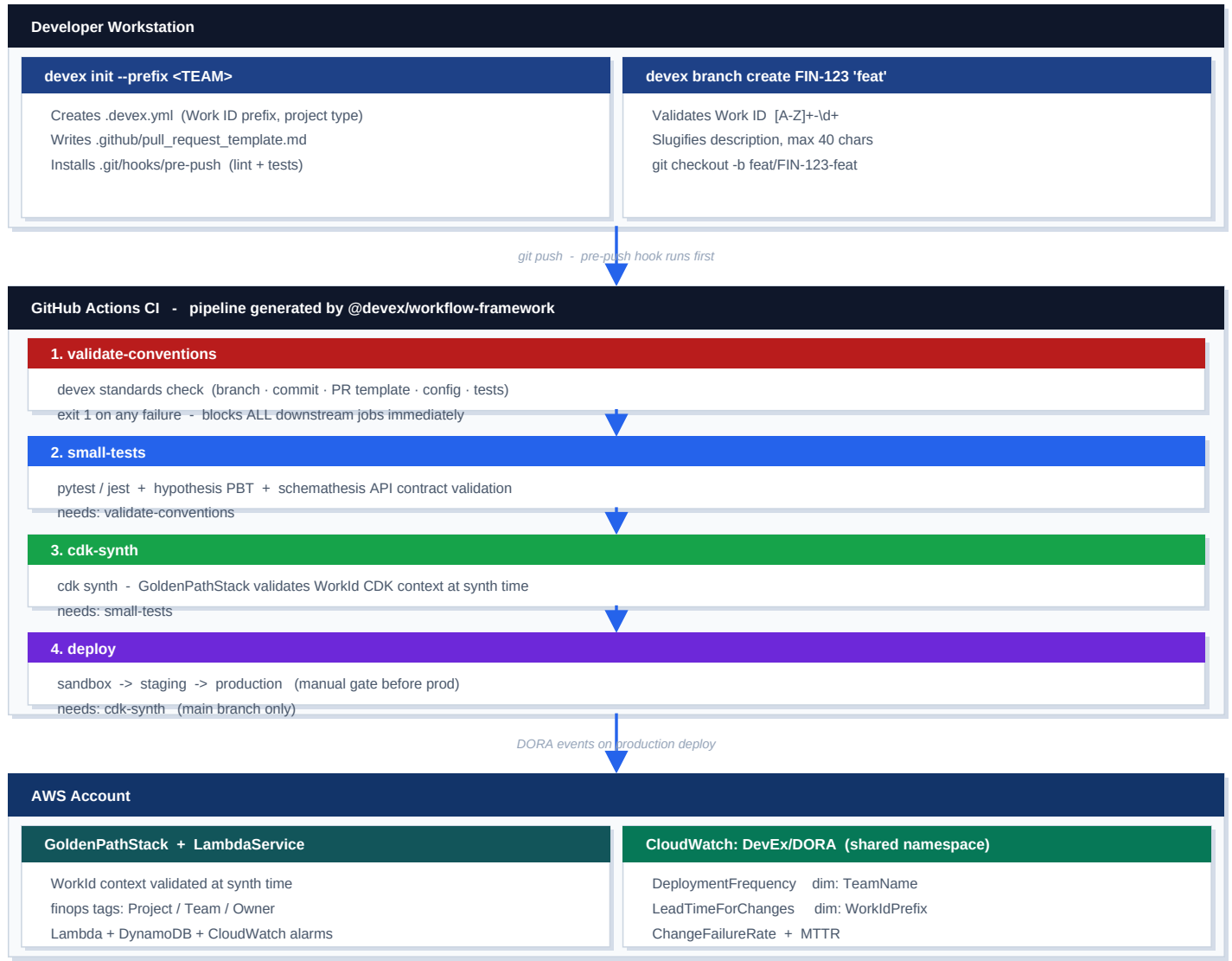
CONTEXT AND PROBLEM STATEMENT

10+ independent engineering teams (Python, TypeScript, Go, Clojure) each maintain their own CI/CD pipelines, branching conventions, and deployment patterns.

- **Inconsistent DORA:** metrics are incomparable across teams
- **Repeated boilerplate:** every team re-implements the same CDK patterns
- **Compliance drift:** branch protection and Work ID enforcement vary by team
- **Slow feedback loops:** defects discovered late in the delivery cycle

Decision: a shared Golden Path that teams adopt voluntarily because it is easier to follow than to work around.

ARCHITECTURE



SHIFT-LEFT STRATEGY

Layer	Mechanism	When
Local	pre-push hook - conventions + lint + unit tests	Before git push
PR	validate-conventions CI job (exit 1 = block pipeline)	Before tests run
AI Review	Amazon Q Developer inline PR comments	On PR open/update
Synth	GoldenPathStack validates WorkId CDK context	Before CloudFormation
Deploy	CloudWatch DORA alarms on metric thresholds	After prod deploy

COMPONENT DECISIONS

- **Component A - Python CLI (devex):**
Typer + Rich for type-safe commands. Distributed via uv for isolated installs on any language team. Validators are pure functions enabling Property-Based Testing with hypothesis.
- **Component B - TypeScript Framework (@devex/workflow-framework):**
github-actions-workflow-ts provides compile-time type safety for GitHub Actions YAML. CDK L3 constructs (GoldenPathStack, LambdaService) reduce 40+ lines of boilerplate to a single call.
- **Convention enforcement - triple layer:**
Work ID validated in (1) branch name, (2) commit message, (3) CDK context at synth time. Making compliance cheaper than non-compliance is the core design principle.

HOMOLOGATION -- ENSURING 10+ TEAMS ADOPT

- **Opt-in, not mandate:**
devex init eliminates toil so adoption is self-motivated - teams are not forced to comply.
- **Reference implementation:**
A fully-onboarded sample microservice is a live Golden-Path demo. Teams clone the pattern rather than read a spec.
- **Metrics visibility:**
devex dora report shows each team their DORA level (Elite/High/Medium/Low). Peer comparison drives adoption.
- **Inner-source model:**
CONTRIBUTING.md documents how to add language support. Teams extend the platform without waiting for the platform team.
- **Amazon Q Developer:**
Automated PR reviews flag convention violations on day one, before any human reviewer comment.

SCALABILITY -- NO PLATFORM BOTTLENECKS

- **No central service:**
CLI and framework are distributed packages (uv / pnpm). No platform API sits in the critical path.
- **Versioned semantic releases:**
Teams pin package versions and upgrade on their schedule without coordinating with the platform team.
- **Extensible pipeline stages:**
generatePRPipeline({ stages: [...] }) lets teams add or remove stages without modifying the framework.
- **CDK as base classes:**
GoldenPathStack and LambdaService are extended, not called as a service. Teams own their CDK stacks.

DORA METRIC CONSISTENCY ACROSS TEAMS

All DORA metrics are emitted to the same CloudWatch namespace (DevEx/DORA) with TeamName and WorkIdPrefix dimensions. A Python team and a Go team are measured identically - the git-log collector and the CloudWatch SDK emitter are fully language-agnostic. A single CloudWatch Insights dashboard can compare any two teams side by side without normalisation.

CONSEQUENCES

Positive outcomes expected after adoption:

- **Reduced onboarding time:**
New teams reach a compliant, observable service within one sprint using devex init and the reference implementation.
- **Comparable DORA baselines:**
All teams report to the same CloudWatch namespace with identical dimensions, enabling organisation-wide DORA dashboards and quarterly performance reviews.
- **Earlier defect detection:**
Convention violations caught at pre-push and PR stages rather than at production deploy, reducing the mean cost of a non-compliant change by an estimated 10x.
- **Platform team unblocked:**
Inner-source model allows teams to contribute language support and CDK constructs independently, preventing the platform team from becoming a bottleneck.

Accepted trade-offs: teams must install uv (Python) or pnpm (Node) to consume the tools. This is a one-time, low-effort setup that is already required by most modern Python and TypeScript projects. The pre-push hook adds 10-30 seconds to every push; this is intentional - catching issues locally is cheaper than catching them in CI or production.